

# FAQ

## FAQ — CLI Coding Agents + Helix

| Field          | Value                                                                                                                                                                         |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Revision       | 1                                                                                                                                                                             |
| Created        | 2026-09-05                                                                                                                                                                    |
| Last modified  | 2026-09-05                                                                                                                                                                    |
| Status         | DRAFT — answers grounded in dated probe evidence; see the source citation on each answer                                                                                      |
| Status summary | Answers the recurring questions about wiring opencode, pi, crush, and Claude Code to Helix’s local LLM surfaces.                                                              |
| Scope          | Companion to <a href="#">README.md</a> and <a href="#">ARCHITECTURE.md</a> in this same directory                                                                             |
| Authority      | Every answer below is either sourced from a live probe run 2026-09-05 or a cited vendor doc — see docs/research/cli_agent_config_schemas/EVIDENCE.md for the raw transcripts. |

---

## Table of contents

1. [Why does my agent list the model but every request fails?](#)
  2. [Why does tool-calling work in one place and not another?](#)
  3. [Why isn’t Claude Code in the list of supported agents?](#)
  4. [Do I need an API key?](#)
  5. [Will wiring Helix overwrite my existing agent config?](#)
  6. [Why does crush create a .crush folder in my project?](#)
  7. [Why did opencode/pi’s request against the Gateway just hang?](#)
  8. [Which surface should I actually use?](#)
-

# 1. Why does my agent list the model but every request fails?

Because listing a model only proves the provider config is syntactically correct and the surface answered a `/v1/models` (or equivalent) call — it says nothing about whether a real request will fit inside that model's context window.

This happened for real on 2026-09-05: the Coder surface (:18434) was running with a 4096-token context window, and a genuine `pi` request — nothing exotic, just the agent's own baseline system prompt — was rejected with:

```
400: request (141440 tokens) exceeds the available context size (4096 tokens)
```

141,440 tokens of system prompt against a 4096-token window. The provider was registered correctly; the server was reachable and answered correctly; the request still could not succeed, because the window was far too small for any real agentic turn.

The Coder surface's context window has since been raised (to 32768 tokens as of 2026-09-05), which should leave much more room, but the underlying lesson stands: **if a model shows up in your agent's model list yet every real turn fails, check the context window of the surface you pointed it at, not the provider config.** Registration proves plumbing, not usability. See [README.md §9](#).

# 2. Why does tool-calling work in one place and not another?

Because it depends on **which Helix surface** you point at, not which model id you ask for. Coder and Gateway serve, as far as the model id is concerned, the same underlying model — but:

- Point an agent at **Coder** (:18434) and a tool-calling request gets a fenced JSON blob pasted into the assistant's plain text reply. There is no structured `tool_calls` field in the response, so the agent's tool-call parser never fires.
- Point the same agent at **Gateway** (:8443) instead, and the response comes back with a real `tool_calls` array and `finish_reason: "tool_calls"` — because the Gateway performs response translation that the raw llama.cpp server behind Coder does not do on its own.

If tool-calling matters for your workflow, use the Gateway surface. See [README.md §3](#) for the fuller explanation, and [ARCHITECTURE.md](#) for where this sits in the overall picture.

### 3. Why isn't Claude Code in the list of supported agents?

Because Claude Code has no mechanism to accept a custom OpenAI-compatible provider at all — this is a constraint on Anthropic's side, confirmed against Claude Code's own official documentation (fetched 2026-09-05):

- The LLM Gateway Protocol documentation (<https://code.claude.com/docs/en/llm-gateway-protocol>) lists exactly three client-selectable API formats: Anthropic Messages, Amazon Bedrock InvokeModel, and Google Agent Platform `rawPredict`. There is no OpenAI Chat Completions row.
- The LLM Gateway documentation (<https://code.claude.com/docs/en/llm-gateway>) states, verbatim: *“Anthropic doesn't endorse, maintain, or audit third-party gateway products, and doesn't support routing Claude Code to non-Claude models through any gateway.”*
- Claude Code's optional model-discovery feature also filters by name: it “keeps an entry when its `id` contains `claude` or `anthropic` anywhere in the string, matched case-insensitively, and ignores the rest” — so even a discovery call against a Helix surface would silently drop every Helix model `id`.

The only conceivable path in is **HelixCode's Anthropic-shaped `/v1/messages` endpoint on `:8080`**, via `ANTHROPIC_BASE_URL` plus an API key. **Whether that actually works was, at the time this FAQ was written, still under verification by a separate work stream — do not assume it does.** Check that stream's findings before relying on it.

### 4. Do I need an API key?

For three of the four Helix surfaces, no:

- **Coder** (`:18434`) — none required.
- **Gateway** (`:8443`) — none required (though you do need TLS trust set up for its self-signed certificate — see [README.md §5](#)).
- **HelixAgent** (`:7061`) — none required.

For the fourth, yes:

- **HelixCode** (`:8080`) — returns HTTP 401 without an API key; one is required.

One quirk to know: `pi` requires you to supply a **non-empty `apiKey` value in its provider config even for a keyless server** — the field cannot be omitted, though its actual value is never checked by Coder, Gateway, or HelixAgent. Any placeholder string (the evidence in this guide uses `"local"`) satisfies `pi`'s own

config validation. `opencode` and `crush` do not have this quirk — you can omit the `key` field entirely for the three keyless surfaces.

## 5. Will wiring Helix overwrite my existing agent config?

It should not, if the wiring is done the way this guide describes: a Helix entry is added as its own named provider block inside the agent's existing config file, alongside whatever else is already there. Adding it should never touch unrelated provider entries, and re-running the same wiring step again should merge into the existing Helix entry rather than create a duplicate.

**Confirm this yourself rather than taking it on faith** — before wiring, note what is currently in your config file (or back it up); after wiring, diff the file and check that everything you had before is still present unchanged, and that only the intended Helix entry was added or updated. See [README.md §8](#) for the fuller discussion, including how to test idempotency by running the wiring step twice.

## 6. Why does crush create a `.crush` folder in my project?

Because `crush`'s data directory defaults to `.crush`, resolved **relative to whatever directory you ran it from** — not a fixed location under your home directory. If you run `crush` from inside a project (this repository included) without overriding that default, it creates `.crush/` right there.

This was observed directly: running `crush` inside this repo's working tree produced a `helix_code/.crush/` directory, which had to be manually removed afterward.

To avoid it, pass explicit flags rather than relying on the default:

```
crush --cwd <some-directory> --data-dir <some-  
directory>/.crush ...
```

or set the equivalent environment variables (`CRUSH_GLOBAL_CONFIG`, `CRUSH_GLOBAL_DATA`) if you want to isolate `crush`'s state away from any project directory entirely.

## 7. Why did opencode/pi's request against the Gateway just hang?

During the research pass behind this guide, both cases ended in a timeout with no output, rather than a clear success or failure — this is recorded honestly as **inconclusive**, not as a documented bug:

- `opencode run --pure --model helixllm-test/...` against the Gateway produced no output before a 280-second timeout (SIGTERM, exit code 143).
- `pi -p` against the HTTPS gateway timed out at 90 seconds, both with and without `NODE_EXTRA_CA_CERTS` set.

Neither run yields a usable verdict on whether generation or TLS trust actually works end-to-end for those two agents against the Gateway. If you hit the same thing, that is a known open question from this research pass, not something this guide can currently explain — re-test it yourself and, if you get a clear result either way, that supersedes what is written here (see the note in `docs/research/cli_agent_config_schemas/EVIDENCE.md`).

## 8. Which surface should I actually use?

As a starting point, based on what has been confirmed so far:

- Need **tool-calling** to work? Use the **Gateway** (:8443) — confirmed real `tool_calls` behavior, but requires TLS trust setup for its self-signed certificate.
- Just need **plain chat completion**, no tools, and want the simplest path (no TLS to configure)? Use the **Coder** surface (:18434).
- Working with **HelixAgent**-specific model ids (`helixagent-llm`, `helixagent-debate`, `helixagent-ensemble`, etc.)? Use :7061 — but note its tool-calling behavior has not been probed, so verify it yourself before depending on it.
- Using **Claude Code**? None of the above are reachable from it at all — see [Q3](#). The HelixCode surface (:8080) is the only candidate, and it is unverified as of this writing.

See [ARCHITECTURE.md](#) for the full picture of which agent can reach which surface.